

DevSecOps Pipeline Security Analyzer

1. Introduction

Modern software development heavily relies on Continuous Integration and Continuous Deployment (CI/CD) pipelines to automate software delivery. While CI/CD improves development speed and operational efficiency, it also introduces security risks such as vulnerable dependencies, infrastructure misconfigurations, and exposed secrets.

To address these challenges, organizations increasingly adopt DevSecOps practices, where security controls are integrated directly into the Software Development Life Cycle (SDLC). This project demonstrates the implementation of a DevSecOps security pipeline using GitHub Actions, Checkov, Trivy, and Terraform.

The solution automatically scans application code, dependencies, infrastructure configurations, and secrets whenever code is pushed to the GitHub repository. Security findings are then reported through GitHub Actions logs before deployment occurs.

2. Project Objective

The main objective of this project is to design and implement an automated security analysis pipeline capable of:

- Detecting Infrastructure-as-Code (IaC) security misconfigurations.
- Identifying vulnerable software dependencies.
- Detecting hardcoded secrets and credentials.
- Integrating automated security scanning into CI/CD workflows.
- Supporting Shift-Left Security practices by identifying risks early in development.

Project Repository

The complete source code, workflow configuration, Terraform files, application code, and project documentation are publicly available on GitHub and can be accessed using the following repository:

GitHub Repository:

<https://github.com/surajs-sudo/devsecops-pipeline-security-analyzer>

Note: This project contains intentionally vulnerable sample code, insecure Terraform configurations, and simulated secrets for educational and security testing purposes. These vulnerabilities were deliberately introduced to demonstrate the effectiveness of DevSecOps security tools such as Checkov and Trivy. The project should not be considered a production-ready implementation.

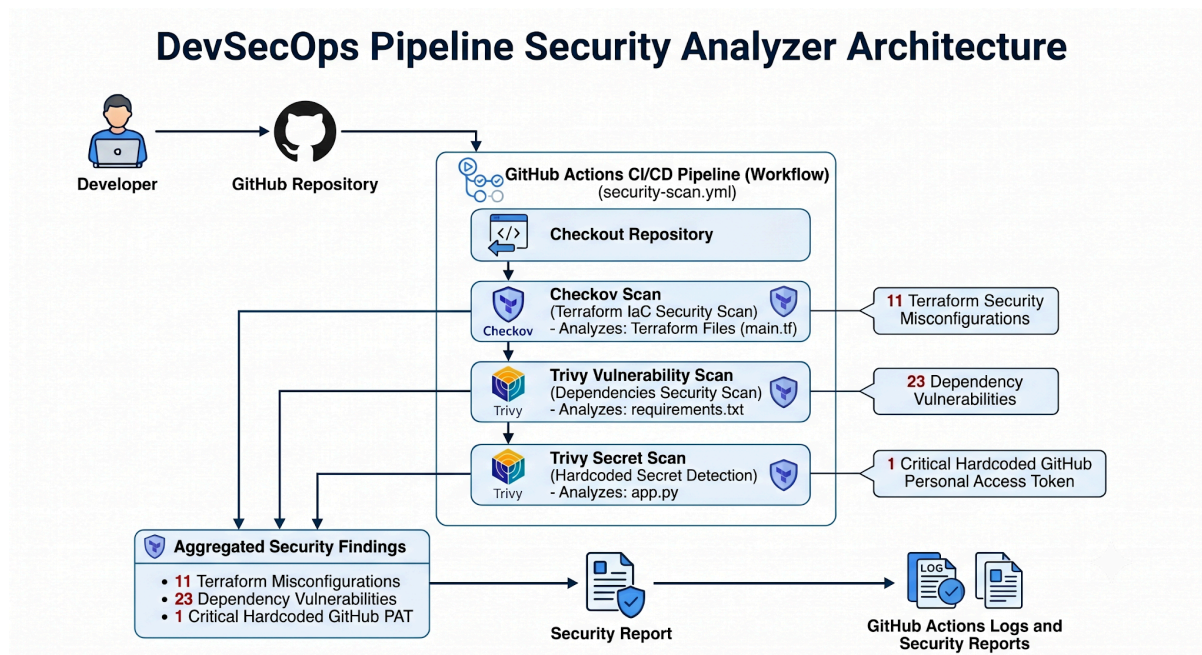
3. Project Architecture

The architecture consists of a GitHub repository connected to a GitHub Actions workflow. Whenever a developer pushes code, the workflow automatically executes multiple security scanning stages.

The workflow performs the following actions:

1. Checkout Repository
2. Execute Checkov Scan
3. Execute Trivy Vulnerability Scan
4. Execute Trivy Secret Scan
5. Aggregate Security Findings
6. Generate Security Reports

Figure 1: DevSecOps Pipeline Security Analyzer Architecture Diagram



Architecture Analysis

The architecture follows a Shift-Left Security approach by integrating security validation directly into the CI/CD pipeline. Instead of identifying security risks after deployment, the solution detects vulnerabilities, infrastructure weaknesses, and exposed secrets during the development stage. This reduces remediation costs and allows vulnerabilities to be fixed before software reaches production environments, improving overall security posture.

The architecture also demonstrates how multiple security tools can be integrated into a single CI/CD workflow to provide comprehensive security coverage.

4. Development Environment

The project repository contains the following components:

- GitHub Actions Workflow
- Terraform Infrastructure Configuration
- Python Application
- Dependency File (requirements.txt)
- Project Documentation

Figure 2: GitHub Repository

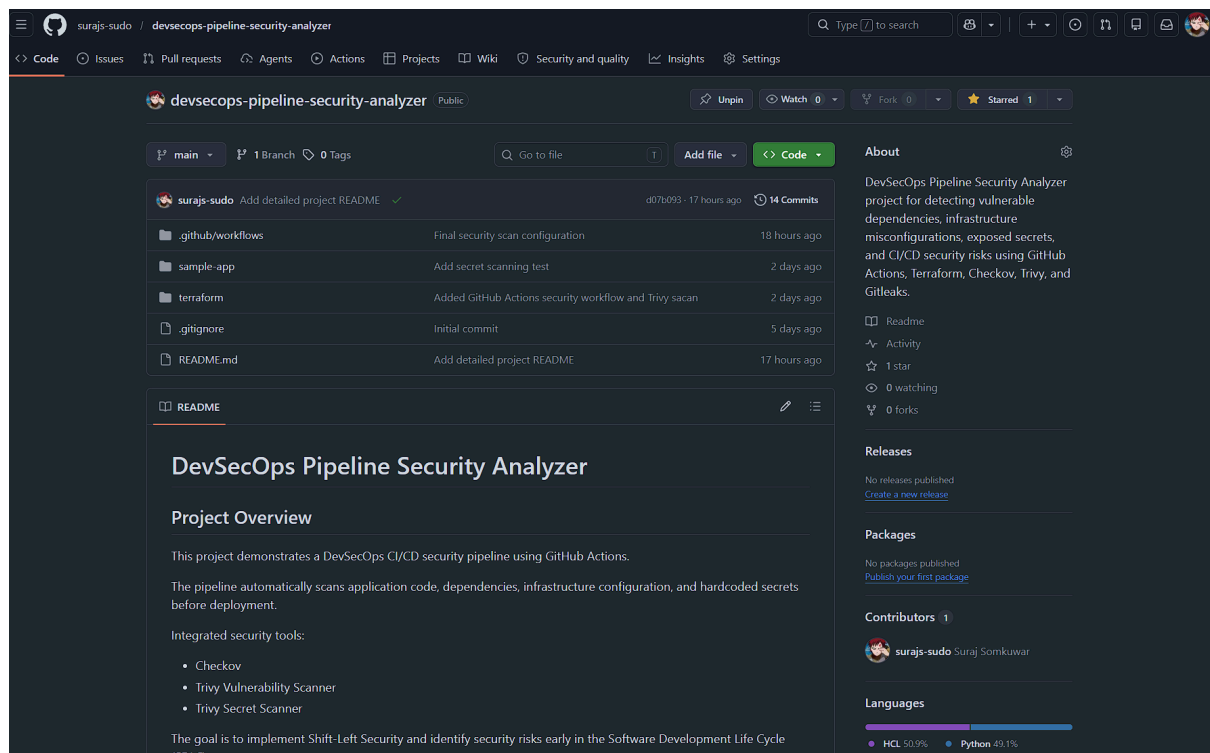
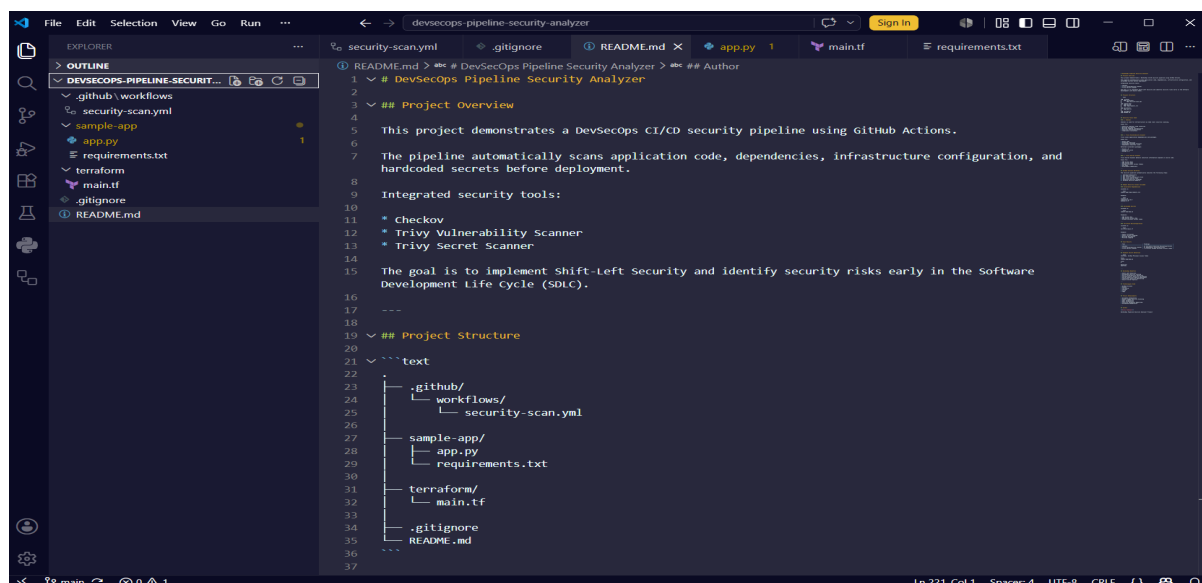


Figure 3: Project Structure in Visual Studio Code



Project Structure Explanation

The repository is organized into multiple folders and files:

- `.github/workflows/security-scan.yml` contains the CI/CD workflow.
- `terraform/main.tf` contains infrastructure configurations.
- `sample-app/app.py` contains the sample application.
- `sample-app/requirements.txt` contains project dependencies.

This structure enables each security tool to analyze specific components of the project.

5. CI/CD Security Workflow

GitHub Actions was used to automate security scanning activities.

The workflow is triggered whenever code is pushed to the main branch. The workflow sequentially executes the following stages:

Stage 1: Repository Checkout

The repository source code is downloaded into the GitHub Actions runner.

Stage 2: Checkov Scan

Checkov analyzes Terraform files and identifies infrastructure security weaknesses.

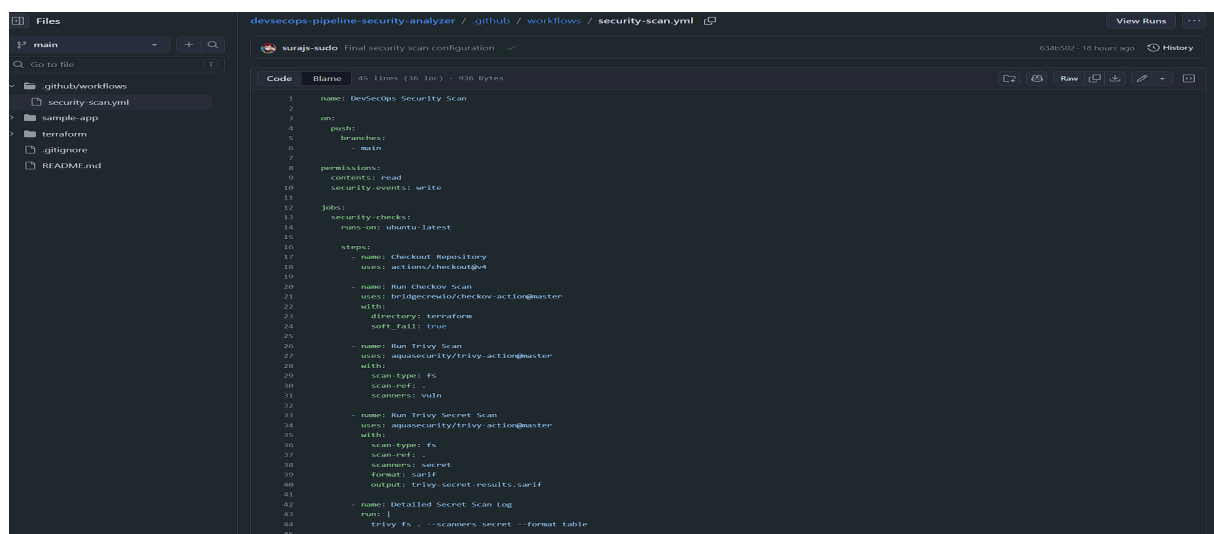
Stage 3: Trivy Vulnerability Scan

Trivy scans application dependencies listed in `requirements.txt` and compares them against known vulnerability databases.

Stage 4: Trivy Secret Scan

Trivy scans application source code for hardcoded credentials and exposed secrets.

Figure 4: GitHub Actions Workflow Configuration (`security-scan.yml`)



The screenshot shows the GitHub Actions workflow configuration for security scanning. The workflow is named 'DevSecOps Security Scan' and is triggered on push to the main branch. It runs on the 'ubuntu-latest' runner. The workflow consists of four main steps: 1. Checkout Repository (using actions/checkout@v4), 2. Run Checkov Scan (using bridgecrew/checkov-action@master, scanning the terraform directory), 3. Run Trivy Scan (using aquasecurity/trivy-action@master, scanning the fs directory), and 4. Run Trivy Secret Scan (using aquasecurity/trivy-action@master, scanning the fs directory for secrets). The workflow also includes a final step to format the results as a table.

```
1 name: DevSecOps Security Scan
2
3 on:
4   push:
5     branches:
6       - main
7
8 permissions:
9   contents: read
10  security-events: write
11
12 jobs:
13   security-checks:
14     runs-on: ubuntu-latest
15
16     steps:
17     - name: Checkout Repository
18       uses: actions/checkout@v4
19
20     - name: Run Checkov Scan
21       uses: bridgecrew/checkov-action@master
22       with:
23         directory: terraform
24         soft-fail: true
25
26     - name: Run Trivy Scan
27       uses: aquasecurity/trivy-action@master
28       with:
29         scan-type: fs
30         scan-ref: .
31         scanners: vuln
32
33     - name: Run Trivy Secret Scan
34       uses: aquasecurity/trivy-action@master
35       with:
36         scan-type: fs
37         scan-ref: .
38         scanners: secret
39         format: sarif
40         output: trivy-secret-results.sarif
41
42     - name: Detailed Secret Scan Log
43       run: |
44         trivy fs . --scanners secret --format table
45
```

Workflow Analysis

The workflow successfully integrates multiple security scanners into a single automated process.

Although security issues were detected, the workflow completed successfully because it was intentionally configured for demonstration purposes. This allowed all security tools to complete their analysis and generate comprehensive reports. In production environments, organizations typically configure security gates that automatically fail the pipeline when Critical or High severity vulnerabilities are detected.

6. Pipeline Execution Results

After pushing code to GitHub, the workflow executed successfully and completed all configured security scans.

Figure 5: GitHub Actions Pipeline Execution Summary

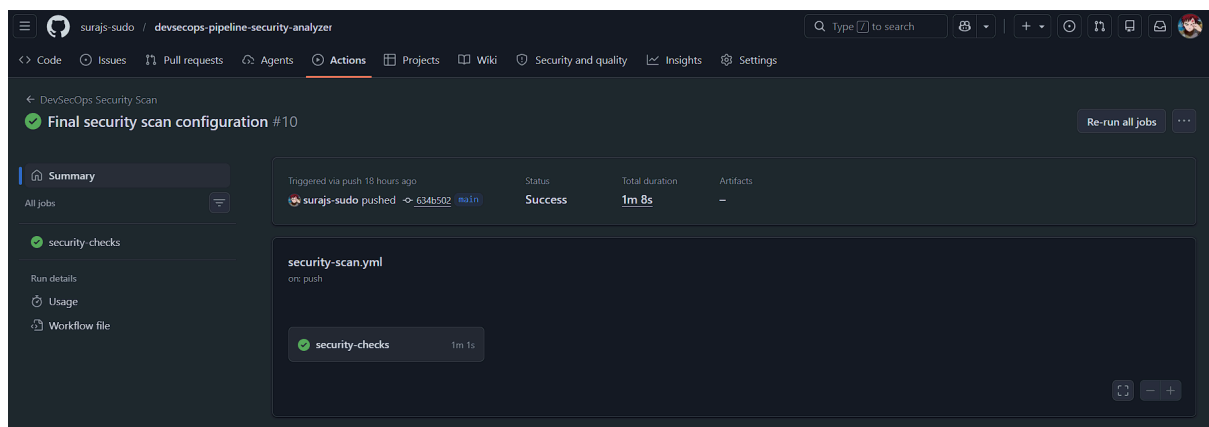
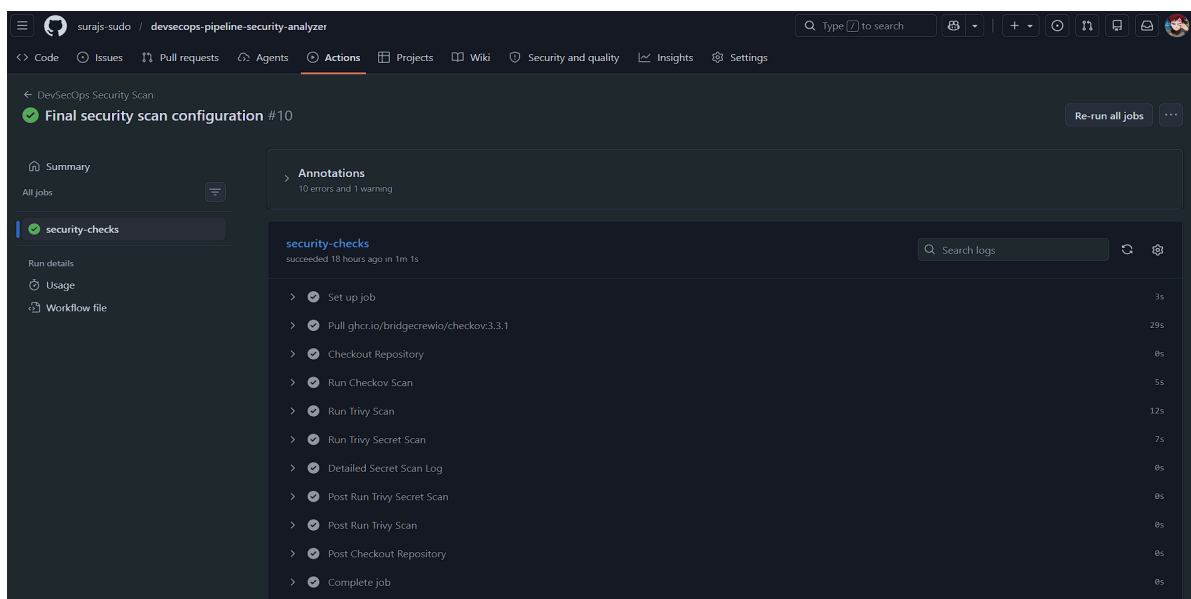


Figure 6: GitHub Actions Pipeline Detailed Execution



Analysis

The successful execution confirms that:

- GitHub Actions workflow is correctly configured.
- Security scanners execute automatically.
- Security findings are generated without manual intervention.
- Results are available within GitHub Actions logs.

Although the GitHub Actions workflow completed successfully, the security scans detected significant security issues. The workflow was intentionally configured using soft-fail behaviour to allow report generation and demonstration purposes. In a production environment, the pipeline should be configured to fail automatically when Critical or High severity vulnerabilities are detected, preventing insecure code from progressing to deployment.

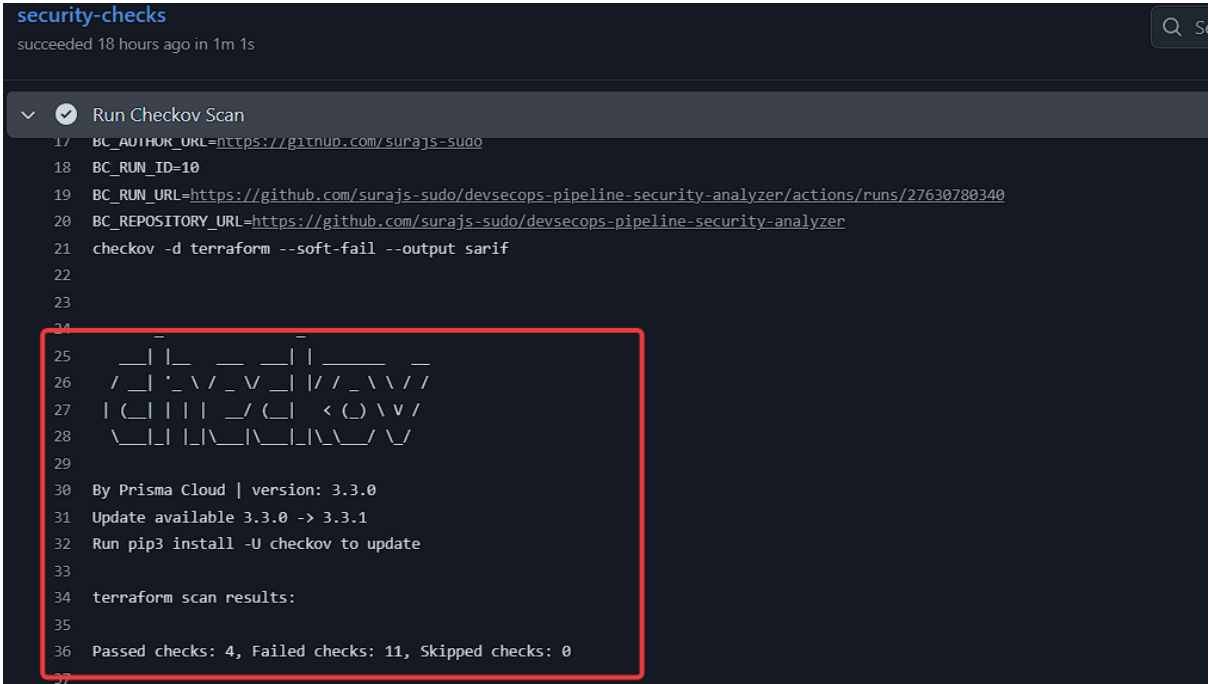
This demonstrates the practical implementation of automated DevSecOps security validation.

7. Infrastructure Security Analysis (Checkov)

Checkov was used to analyze Terraform infrastructure configurations.

The Terraform configuration intentionally contains insecure settings to demonstrate the effectiveness of infrastructure security scanning.

Figure 7: Checkov Scan Summary



```
security-checks
succeeded 18 hours ago in 1m 1s

Run Checkov Scan
17 BC_AUTHOR_URL=https://github.com/surajs-sudo
18 BC_RUN_ID=10
19 BC_RUN_URL=https://github.com/surajs-sudo/devsecops-pipeline-security-analyzer/actions/runs/27630780340
20 BC_REPOSITORY_URL=https://github.com/surajs-sudo/devsecops-pipeline-security-analyzer
21 checkov -d terraform --soft-fail --output sarif
22
23
24
25
26
27
28
29
30 By Prisma Cloud | version: 3.3.0
31 Update available 3.3.0 -> 3.3.1
32 Run pip3 install -U checkov to update
33
34 terraform scan results:
35
36 Passed checks: 4, Failed checks: 11, Skipped checks: 0
```

Figure 8: Example Terraform Misconfiguration Detected by Checkov

```
security-checks
succeeded 18 hours ago in 1m 1s

▼ Run Checkov Scan
52
53 Guide: https://docs.prismacloud.io/en/enterprise-edition/policy-reference/aws-policies/s3-policies/s3-2-acl-write-permissions-every
54 Check: CKV_AWS_54: "Ensure S3 bucket has block public policy enabled"
55 FAILED for resource: aws_s3_bucket_public_access_block.public_access
56 Error: File: /main.tf:5-12
57 Guide: https://docs.prismacloud.io/en/enterprise-edition/policy-reference/aws-policies/s3-policies/bc-aws-s3-20
58
59 5 | resource "aws_s3_bucket_public_access_block" "public_access" {
60 6 |   bucket = aws_s3_bucket.public_bucket.id
61 7 |
62 8 |   block_public_acls = false
63 9 |   block_public_policy = false
64 10 |   ignore_public_acls = false
65 11 |   restrict_public_buckets = false
66 12 | }
67 Check: CKV_AWS_53: "Ensure S3 bucket has block public ACLS enabled"
68 FAILED for resource: aws_s3_bucket_public_access_block.public_access
69 Error: File: /main.tf:5-12
70 Guide: https://docs.prismacloud.io/en/enterprise-edition/policy-reference/aws-policies/s3-policies/bc-aws-s3-19
71
72 5 | resource "aws_s3_bucket_public_access_block" "public_access" {
73 6 |   bucket = aws_s3_bucket.public_bucket.id
74 7 |
75 8 |   block_public_acls = false
76 9 |   block_public_policy = false
77 10 |   ignore_public_acls = false
78 11 |   restrict_public_buckets = false
79 12 | }
```

Findings

Checkov reported:

- 11 Failed Checks
- 4 Passed Checks

Examples of detected issues include:

- Public Access Restrictions Disabled
- Public Bucket Policies Allowed
- Public ACLs Enabled

Analysis

The identified findings indicate that cloud resources could be publicly accessible and insufficiently protected.

If such configurations were deployed in production environments, attackers could potentially gain unauthorized access to sensitive cloud resources and data.

These findings demonstrate the importance of validating Infrastructure-as-Code before deployment.

8. Dependency Vulnerability Analysis (Trivy)

Trivy was used to scan application dependencies defined in requirements.txt.

The scanner compares installed package versions against known Common Vulnerabilities and Exposures (CVEs).

Figure 9: Trivy Vulnerability Scan Summary

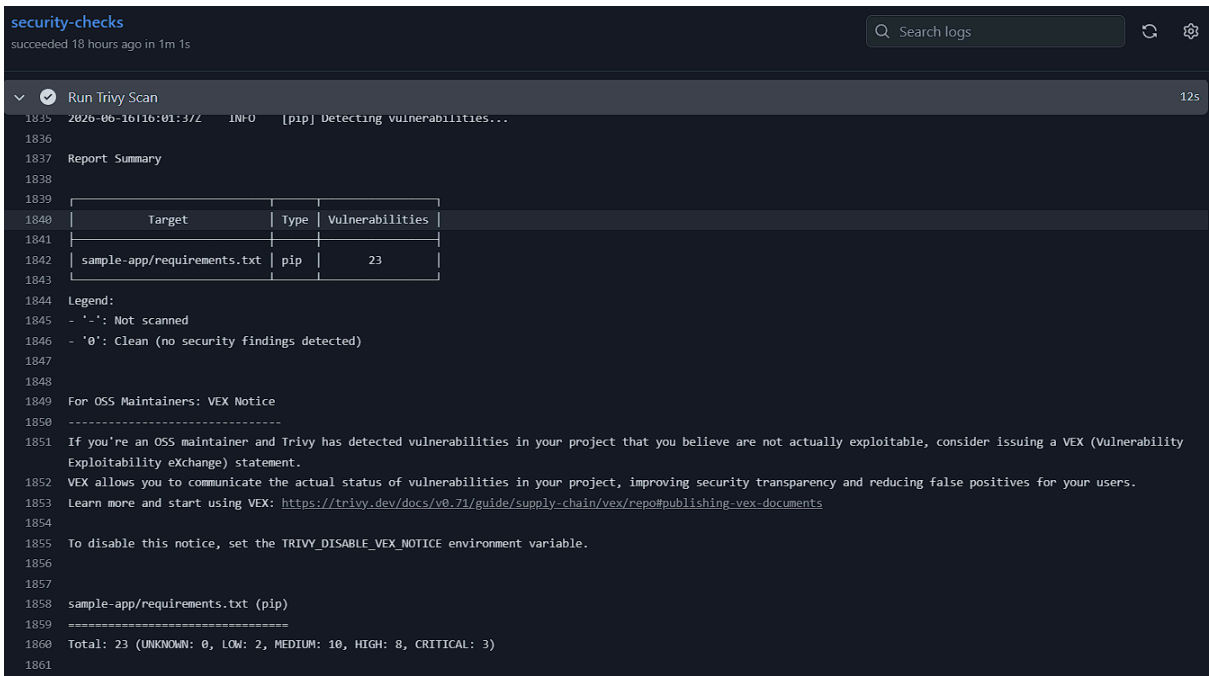
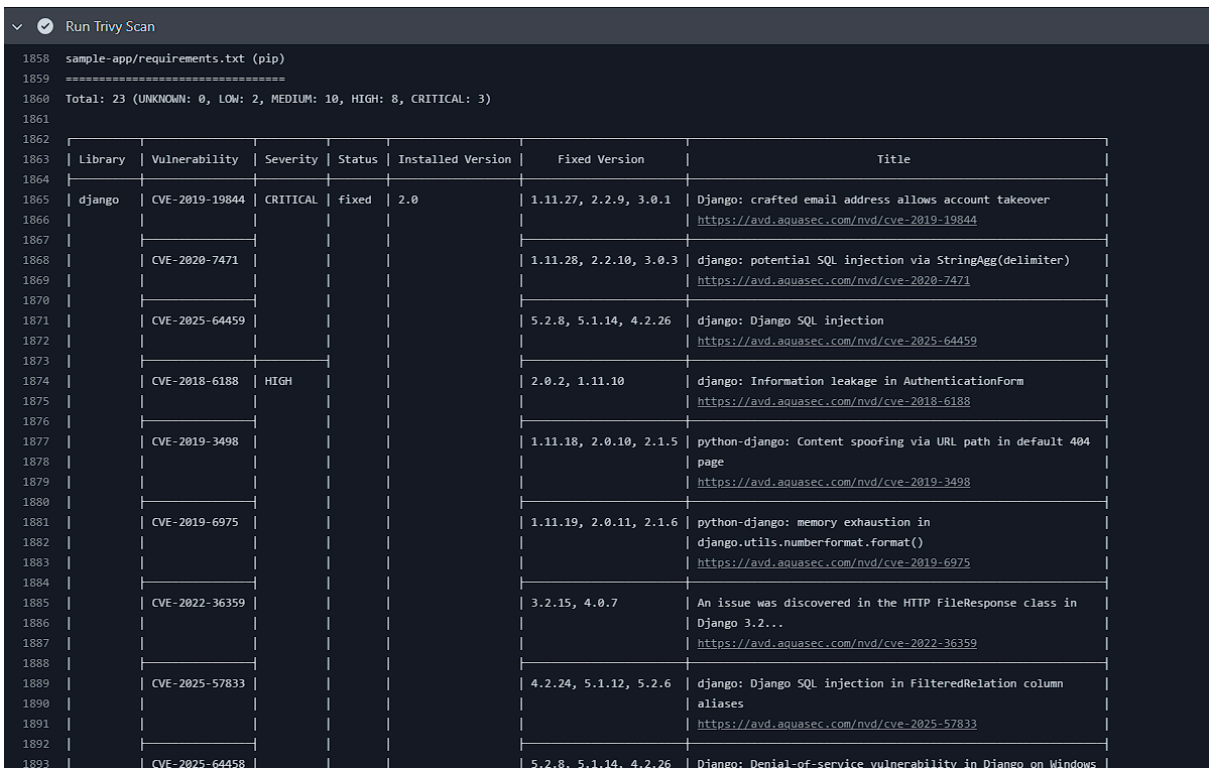


Figure 10: Example CVE Findings



Findings

Trivy detected:

- Total Vulnerabilities: 23
- Critical: 3
- High: 8
- Medium: 10
- Low: 2

Several vulnerabilities were identified in Django packages.

Analysis

The presence of Critical and High severity vulnerabilities highlights the security risks associated with outdated dependencies.

If exploited, these vulnerabilities could result in:

- Remote Code Execution
- Privilege Escalation
- Information Disclosure
- Denial of Service Attacks

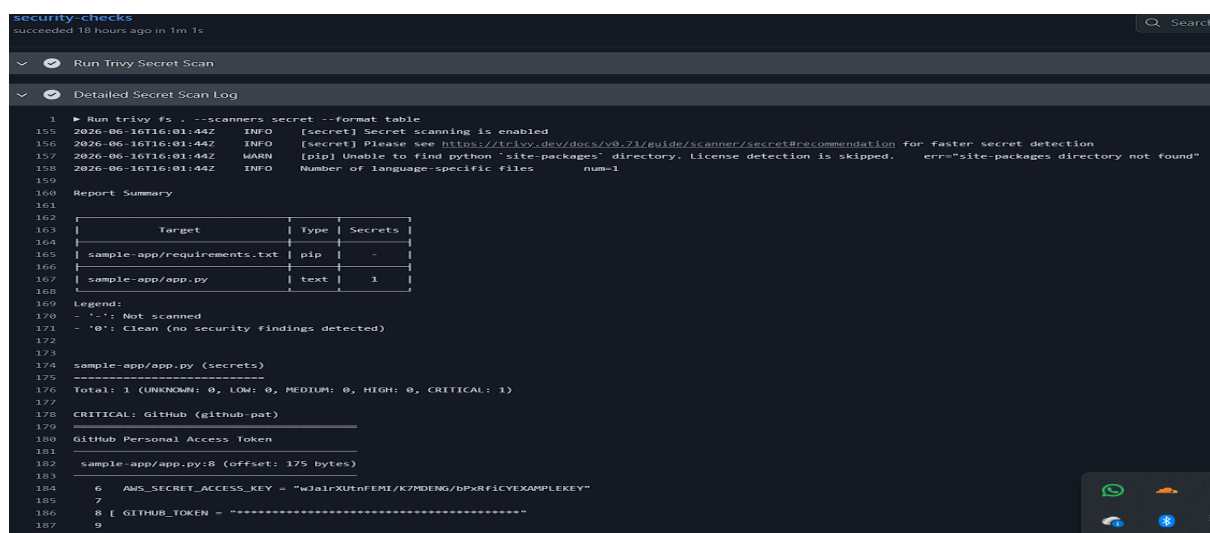
This demonstrates why dependency scanning is a critical component of modern DevSecOps pipelines.

9. Secret Detection Analysis

Hardcoded credentials represent one of the most common causes of security breaches.

Trivy Secret Scanner was used to identify exposed credentials within source code files.

Figure 11: Trivy Secret Detection Result



The screenshot shows a terminal window with the following content:

```
security-checks
succeeded 18 hours ago in 1m 1s

Run Trivy Secret Scan
Detailed Secret Scan Log

1 ▶ Run trivy fs --scanners secret --format table
155 2026-06-16T16:01:44Z INFO [secret] Secret scanning is enabled
156 2026-06-16T16:01:44Z INFO [secret] Please see https://trivy.dev/docs/v0.71/guide/scanner/secret#recommendation for faster secret detection
157 2026-06-16T16:01:44Z WARN [pip] Unable to find python 'site-packages' directory. License detection is skipped. err="site-packages directory not found"
158 2026-06-16T16:01:44Z INFO Number of language-specific files num=1
159
160 Report Summary
161
162
163 | Target | Type | Secrets |
164 |-----|-----|-----|
165 | sample-app/requirements.txt | pip | - |
166 |-----|-----|-----|
167 | sample-app/app.py | text | 1 |
168 |-----|-----|-----|
169 Legend:
170 - '?': Not scanned
171 - '0': Clean (no security findings detected)
172
173 sample-app/app.py (secrets)
174 =====
175 Total: 1 (UNKNOWN: 0, LOW: 0, MEDIUM: 0, HIGH: 0, CRITICAL: 1)
176
177 CRITICAL: GitHub (github-pat)
178 =====
179 GitHub Personal Access Token
180
181 sample-app/app.py:8 (offset: 175 bytes)
182
183 6 AWS_SECRET_ACCESS_KEY = "wJa1rXUtnFEMI/K7MDENG/bpRfjCYEXAMPLEKEY"
184 7
185 8 [ GITHUB_TOKEN = "*****"
186 9
187
188
```

Findings

The scanner detected:

- 1 Critical Hardcoded GitHub Personal Access Token (PAT)

Location:

- sample-app/app.py

Analysis

The detected GitHub Personal Access Token represents a critical security risk.

If exposed publicly, attackers could:

- Access source code repositories
- Modify project files
- Download sensitive data
- Abuse repository permissions

Organizations should never store credentials directly in source code. Instead, secure secret management solutions such as GitHub Secrets, AWS Secrets Manager, or Azure Key Vault should be used.

10. Security Findings Summary

The security assessment identified three major categories of security risks:

Infrastructure Security Findings

- 11 Terraform Misconfigurations

Dependency Security Findings

- 23 Vulnerable Dependencies

Secret Exposure Findings

- 1 Critical Hardcoded GitHub Personal Access Token

These results demonstrate that security issues can exist across multiple layers of modern software systems, including infrastructure, applications, and credentials.

11. Recommendations

Based on the findings, the following recommendations are proposed:

Recommendation 1

Update vulnerable dependencies regularly and automate dependency monitoring.

Recommendation 2

Apply Infrastructure-as-Code security best practices and remediate Terraform misconfigurations before deployment.

Recommendation 3

Remove hardcoded secrets from source code and use secure secret management solutions.

Recommendation 4

Configure CI/CD security gates that automatically fail builds when Critical or High severity findings are detected.

Recommendation 5

Perform security scanning continuously throughout the software development lifecycle.

12. Conclusion

This project successfully implemented a DevSecOps Pipeline Security Analyzer using GitHub Actions, Checkov, Trivy, and Terraform.

The automated workflow successfully detected:

- 11 infrastructure security misconfigurations.
- 23 dependency vulnerabilities.
- 1 critical hardcoded GitHub Personal Access Token.

The project demonstrates the effectiveness of integrating security directly into CI/CD pipelines and supports Shift-Left Security principles. By identifying security weaknesses before deployment, organizations can significantly reduce risk and improve overall software security posture.

Disclaimer

This project was developed for educational and assessment purposes. The vulnerable dependencies, insecure Terraform configurations, and exposed credentials included in the repository were intentionally created to demonstrate the capabilities of automated DevSecOps security scanning tools. No production systems, real credentials, or operational environments were used.

References

GitHub Repository:

<https://github.com/surajs-sudo/devsecops-pipeline-security-analyzer>

Checkov Documentation:

<https://www.checkov.io>

Trivy Documentation:

<https://trivy.dev>

GitHub Actions Documentation:

<https://docs.github.com/actions>